

EXPERIMENT 1 — 8-PUZZLE PROBLEM

AIM

To write a Python program to solve the **8-Puzzle problem using Breadth-First Search (BFS)**.

ALGORITHM

1. Represent the 8-puzzle as a 3×3 matrix.
2. Define the initial state and goal state.
3. Use a queue to store puzzle states.
4. Find the position of the blank tile 0.
5. Move the blank tile up, down, left, or right.
6. Store unvisited states in the queue.
7. Continue until the goal state is reached.
8. Display the solution path.

PYTHON PROGRAM

```
from collections import deque
```

```
start = (1, 2, 3,  
         4, 0, 6,  
         7, 5, 8)
```

```
goal = (1, 2, 3,  
        4, 5, 6,  
        7, 8, 0)
```

```
queue = deque([(start, [])])
```

```
visited = {start}
```

```
while queue:
```

```
    state, path = queue.popleft()
```

```
    if state == goal:
```

```
        print("Solution:")
```

```
for s in path + [state]:
```

```
    print(s[0:3])
```

```
    print(s[3:6])
```

```
    print(s[6:9])
```

```
    print()
```

```
break
```

```
zero = state.index(0)
```

```
row, col = divmod(zero, 3)
```

```
moves = [(1, 0), (-1, 0), (0, 1), (0, -1)]
```

```
for dr, dc in moves:
```

```
    nr = row + dr
```

```
    nc = col + dc
```

```
if 0 <= nr < 3 and 0 <= nc < 3:
```

```
    new_zero = nr * 3 + nc
```

```
    new_state = list(state)
```

```
    new_state[zero], new_state[new_zero] = \
```

```
        new_state[new_zero], new_state[zero]
```

```
    new_state = tuple(new_state)
```

```
if new_state not in visited:
```

```
    visited.add(new_state)
```

```
    queue.append(
```

```
        (new_state, path + [state])
```

```
    )
```

OUTPUT :

Solution:

```
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)
```

```
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
```

```
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

RESULT

Thus, the **8-Puzzle problem** was successfully solved using **Breadth-First Search in Python**.